

Figure 3: (a) Average performance with dummy rationale in Shakespeare’s Sonnet. (b) Average performance with dummy rationale in countdown list. (c) Effect of rationale frontloading. All scores represent mean accuracies across six LLMs.

It is important to acknowledge that this "dummy rationale" approach is not without its limitations. Producing Shakespeare-like text or a procedural countdown is not identical to producing a task-relevant rationale and may introduce process differences unrelated to contextual distance (e.g., stylistic or format-specific generation effects), which is distinct from the structural effect of contextual distance. Despite these potential confounders, these tasks were chosen for their highly controllable output length and low semantic overlap with the pattern-recognition problems, allowing us to introduce a lengthy, unrelated text block that structurally mimics a CoT rationale. Therefore, while not a perfect control, the consistent performance degradation observed in this experiment provides strong evidence that is highly consistent with our contextual distance hypothesis.

Note that we adopt **in-response** dummy rationales, as this approach preserves the relative positioning between the chat template and the rationale, consistent with standard CoT prompting. However, an alternative format that involving **in-prompt** dummy rationales has also been proposed (Lanham et al., 2023). Experimental results demonstrate that both methods provide evidence supporting the contextual distance curse (see Appendix B.1.2).

Rationale Frontloading Experiment To preserve CoT semantics while eliminating contextual distance, we first elicited CoT rationales from LLMs for each test instance. Specifically, for each instance, we obtained its rationale under the regular CoT setting, then reused the same rationale (with the final answer removed to prevent leakage) in the front-loading condition by placing it before the demonstrations and query. Thus, the rationale content is identical across conditions, and only its position in the prompt (i.e., the contextual distance) differs. This approach ensures that the model has access to the same reasoning content, without separating demonstrations from the answer prediction.

The experimental results are presented in Figure 3. From the dummy rationale experiment, we observe that LLM performance generally declines as contextual distance increases. The only exception occurs in the countdown task on the COGS dataset, where LLMs frequently refuse to generate dummy rationales when instructed to count down from 200 or 400. From the rationale frontloading experiment, we find that performance substantially improves when rationales are prepended to the in-context demonstrations. These results provide significant evidence supporting Hypothesis 1. However, we also note that dummy rationales outperform CoT (on MiniARC and COGS), even at greater contextual distances, while frontloaded rationales still underperform relative to direct answering. These observations suggest that contextual distance alone does not fully account for the observed "curse". Additional limitations inherent to CoT itself must also contribute to its inefficacy.

Findings: Hypothesis 1 is validated; however, it does not fully explain the CoT curse.

5.2 Pattern inference vs. execution: two stages of failure

Chain-of-Thought in pattern-based in-context learning is commonly regarded as a two-stage process: first, LLMs infer the underlying pattern or rule from the provided demonstration pairs, and second, they apply this

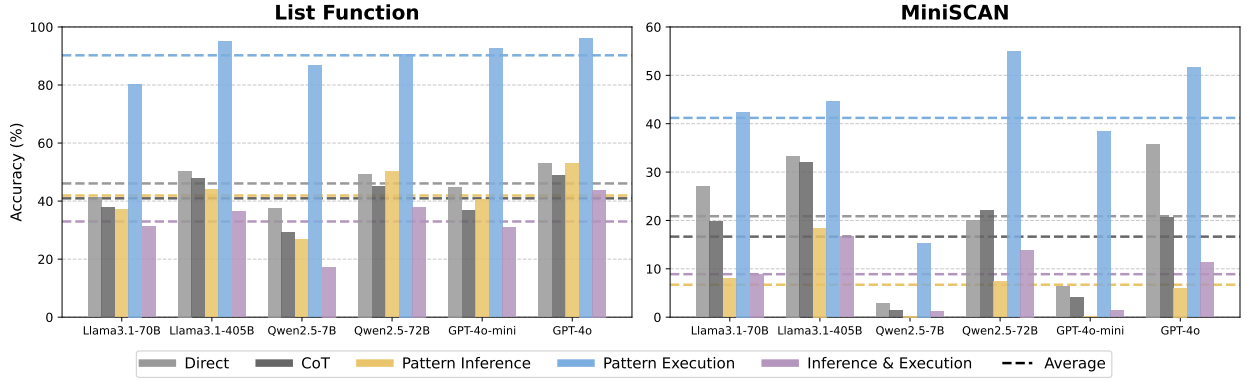


Figure 4: Performance comparison of pattern inference and execution across two benchmarks (List Function and MiniSCAN) and six LLMs.

inferred pattern to generate predictions for test instances (Liu et al., 2024; Zheng et al., 2025a). Given the observed deficiencies of CoT in our experiments, we propose two hypotheses to dissect the potential sources of this failure:

Hypothesis 2. LLMs struggle to infer underlying patterns from demonstrations when using CoT.

Hypothesis 3. LLMs struggle to apply inferred patterns to test instances when using CoT.

To rigorously test these hypotheses, we designed a two-phase experiment to independently evaluate LLM performance across both stages: pattern inference and pattern execution. For this analysis, we selected two datasets—List-Function and MiniSCAN—which allow for precise evaluation against ground-truth patterns. In the pattern inference stage, we assessed whether LLMs could correctly infer the underlying pattern (e.g., a Python function or symbolic rule) from input-output pairs in demonstrations. In the pattern execution stage, we evaluated their ability to apply the ground-truth pattern to test instances. We further evaluated "Inference & Execution" performance by collecting the inferred pattern (without format constraints) from CoT reasoning and directly providing it for pattern execution. This setup aims to further validate the consistency between the original CoT process and a separate inference-then-execution mechanism.

The experimental results are depicted in Figure 4. Across both datasets, we observe that LLM performance in pattern inference consistently falls below that of pattern execution. This disparity suggests that the primary challenge for LLMs using CoT lies in accurately deducing the underlying rules from demonstration pairs. In contrast, their ability to execute a given pattern appears relatively stronger, though still imperfect.

On the other hand, we also observe that the overall performance of "Inference & Execution" remains significantly lower than that of the original CoT across both datasets. This pronounced disparity—combined with the observations above—suggests that CoT often produces correct answers despite incorrect pattern inference in numerous cases (we provide further case studies in Appendix B.4 for an intuitive demonstration). In other words, rather than relying solely on explicit pattern execution for the test instance, the LLM’s answer generation under CoT appears to be implicitly influenced by the in-context demonstrations, which improves the final performance. This interpretation is supported by experimental results in Appendix B.5.

This challenges the simplistic assumption of CoT as a strictly two-stage mechanism. Instead, these results suggest the presence of implicit reasoning mechanisms within CoT, whereby LLMs leverage latent pattern recognition and execution—akin to the processes underlying direct answering—to compensate for shortcomings in explicit inference and execution.

Findings: Our results indicate that pattern inference (H2) constitutes a more significant bottleneck for LLMs than pattern execution (H3): the primary failure point lies in inducing the correct rule from demonstrations. Nonetheless, our evidence also suggests that implicit mechanisms beyond explicit rule-following may also contribute substantially to CoT performance.

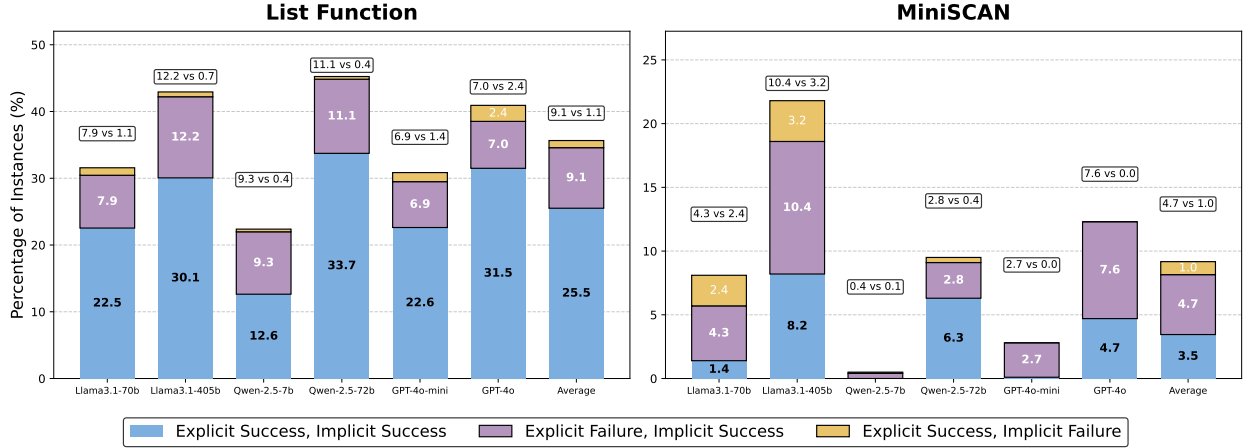


Figure 5: Decomposition of CoT success: contributions from explicit and implicit reasoning.

5.3 The Explicit-Implicit Hybrid Mechanism: uncovering divergent answers and rationales

Findings from the preceding analyses provide compelling empirical evidence for a new conceptualization of CoT in in-context learning: the **Explicit-Implicit Hybrid Mechanism**. This perspective posits that the final prediction in CoT emerges from a composite process involving both explicit pattern inference and execution (articulated through the CoT rationale) and implicit pattern recognition and execution (latent reasoning akin to direct answering). The observed discrepancy between the poor performance of explicit pattern inference (Section 5.2) and the relatively high accuracy of CoT predictions suggests that implicit mechanisms may compensate for deficiencies in the explicit reasoning process. Building on this insight, we formulate and investigate a hypothesis that characterizes this dual process:

Hypothesis 4. In pattern-based ICL, CoT predictions can be modeled as arising from a dual process of explicit and implicit reasoning. This process appears to be asymmetric: implicit reasoning is the dominant driver of successful predictions, compensating for the frequent ineffectiveness of explicit reasoning.

It is challenging to precisely disentangle the effects of implicit and explicit reasoning within a single CoT process. To investigate their relative contributions, we adopt an analytical framework that relies on operationalizing these concepts with proxies.

Analytical Premises: First, we treat **direct answering** (i.e., prediction without an intermediate rationale) as a functional proxy for a purely *implicit reasoning* process. While we acknowledge that direct answering may involve its own un-verbalized reasoning, it serves as a baseline for the model’s performance without explicit rationalization. Second, we treat correct **pattern inference**³ from demonstrations as a proxy for successful *explicit reasoning*. We then analyze instances where the LLM with CoT solved a given problem i successfully, attributing the success as follows:

1. **Implicit Contribution:** We attribute the success on instance i to implicit reasoning if the model also solves i using direct answering but fails to infer the correct pattern from the demonstrations (i.e., explicit reasoning fails).
2. **Explicit Contribution:** Conversely, we attribute the success to explicit reasoning if the model correctly infers the pattern but fails to solve i using direct answering (i.e., implicit reasoning fails).

The results of this analysis are illustrated in Figure 5, where instances of implicit-only and explicit-only contributions are shown in purple and yellow, respectively. Across both datasets, the percentage of cases where implicit reasoning drives CoT success is substantially higher—by $7.5\times$ on List Function and $3.6\times$

³Using “pattern inference” alone provides a conservative upper bound for the contribution of explicit reasoning, thereby strengthening our conclusion.